

Measuring Autonomous Agent Validity: A Dynamical Trust Formula for AI-Native Development Pipelines

Elisha Sam Peter Prabh Preethi Rangamma

Abstract

Strategic Context

Trimble is moving toward AI-native engineering: software agents can now implement features, create pull requests, run tests, and respond to review feedback. The expected productivity gain is substantial. This shift changes the developer's role from producing every line of code to controlling and reviewing increasingly autonomous work.

The Trust Gap in AI-Native Engineering

As agent-authored pull-request volume grows, the limiting resource is no longer code generation; it is the ability of engineers to determine whether an agent's output is safe, complete, and worth merging. Recent longitudinal research reports that AI-authored pull requests take approximately 20% longer from first human review to merge, even while review depth declines under increased workload [1]. Existing benchmarks primarily score models on fixed tasks. They do not tell a Trimble team whether its particular combination of agent, repository, MCP servers, rules, tests, QA, and review gates produces trustworthy output.

Proposed Autonomous Development System

This work shifts attention from the model alone to the system around it. The setup is implemented with Cursor Cloud Automations in the Modus Web Components project, Trimble's open-source component library. A developer starts work from GitHub; cloud agents then implement, QA, repair, and report on the change.

The agents coordinate through simple GitHub handoffs. For example, a label can signal that an issue is ready for development, that QA has failed, or that a repaired pull request must be checked again. This avoids a separate orchestration server while making every handoff a timestamped measurement for the experiment. The controlled Modus experiment is in progress; only measurements collected from it will be reported as project results.

Mathematical Framework for Agent Validity

To evaluate this setup, we derive a dynamical validity model. The model separates the forces that restore trust from those that erode it during an autonomous run.

$$\frac{dV}{dt} = \underbrace{(1 - V(t))(\alpha T(t) + \beta M(t) + \gamma \rho(t) \iota)}_{\text{Systematic Recovery}} - \underbrace{V(t)(C \cdot \sigma_{\text{spec}} \cdot H_c(t) \cdot O(t))}_{\text{Systematic Decay}}$$

Systematic Recovery

$$R(t) = \alpha T(t) + \beta M(t) + \gamma \rho(t) \iota$$

- $T(t)$ Deterministic testing coverage: unit, integration, accessibility, and visual checks.
- $M(t)$ Human-alignment opportunity: a clarification or needs-human escalation.
- $\rho(t)$ Checkpointing frequency: small, reversible commits that bound error scope.
- ι (iota) Tool idempotency: retrying a tool action does not corrupt the environment.
- α, β, γ Fitted weights: the measured recovery value of each safeguard.

Systematic Decay

$$D(t) = C \cdot \sigma_{\text{spec}} \cdot H_c(t) \cdot O(t)$$

- C Blast radius: dependency coupling and reach of the files changed.
- σ_{spec} Specification ambiguity: how much independently generated plans disagree.
- $H_c(t)$ Contextual entropy: context saturation amplified by tool failures and error loops.
- $O(t)$ Diff opacity: review burden from change size, complexity, and file spread.
Higher task complexity increases decay pressure unless recovery scales with it.

System validity, written in the main equation as $V(t)$, is the proportion of task requirements currently satisfied. We observe it as the pass fraction of a verifier at each agent commit. Recovery is generated by deterministic testing, human-alignment opportunities, checkpointing, and safe-to-retry tools. Decay is generated by blast radius, specification ambiguity, contextual entropy, and diff opacity: the conditions under which valid work is more likely to become incorrect or too costly to review.

The derivation starts with the compounding-error property of multi-step agents: if each step has a small probability of failure, the probability of an entirely correct trajectory declines exponentially with the number of steps. Taking the continuous-time limit produces a decay rate. Adding recovery for errors caught and repaired by the surrounding setup produces the equation above.

Equilibrium Validity

$$V^* = \frac{R}{R + D}$$

At equilibrium, recovery strength R must scale with decay pressure D to maintain a target level of trust.

A stronger model is useful; a sufficiently strong surrounding setup is necessary.

Here, R is recovery strength and D is task decay pressure. The implication is practical: to maintain a target level of trust, the strictness of the setup must scale with the complexity and risk of the task. Selecting a stronger model is secondary to designing adequate recovery mechanisms.

Experimental Validation and Adoption Model

The formula is actionable rather than descriptive. We treat each setup addition as a measurable factor: MCP context, repository rules, Playwright-based QA, accessibility gates, visual regression, mutation testing, CI gates, and review agents can each be enabled or disabled in controlled runs. Their contribution is estimated from the validity difference between otherwise comparable runs. The resulting trust gate labels a pull request as *merge-worthy*, *review carefully*, or *abandon*, and a live dashboard shows the validity trajectory and the recovery and decay terms behind that verdict.

Other teams can use the framework at three levels. Level 0 is a drop-in pull-request trust gate using diff statistics, CI status, and GitHub event history with factory-calibrated weights. Level 1 instruments an existing agent pipeline using label conventions and telemetry. Level 2 calibrates the formula to a specific repository by selecting 5–10 representative tasks, using an AI agent to generate verifiers, and comparing baseline and one-factor-off runs. This makes statements such as “this MCP context server adds [X] validity on medium-complexity tasks” measurable rather than anecdotal.

Cloud-only operating-model note: In parallel with the Modus validity experiment, we are evaluating an iPad-based operating surface for Cursor Cloud Automations. A developer creates, approves, observes, and intervenes through GitHub, while cloud agents run

development, QA, and repair in cloud VMs. This is a separate usability and governance workstream; it shares the automation setup and telemetry but does not alter the formula-fitting experiment.

Sources

1. *AI Writes Faster Than Humans Can Review: A Longitudinal Study of an Enterprise “2×” Mandate*, arXiv:2607.01904, 2026. The study reports approximately 20% longer time from first human review to merge for comparable AI-authored pull requests.
2. METR, *Measuring AI Ability to Complete Long Tasks*, arXiv:2503.14499, 2025. Used as the basis for task-complexity stratification and the compounding-error motivation.
3. Sierra Research, *τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*, arXiv:2406.12045, 2024. Source of the pass-to-the-k consistency measure used in the extended methodology.